

Práctica 5: SISTEMAS DE ECUACIONES LINEALES

Objetivos

1. Trabajar computacionalmente las técnicas de resolución numérica para sistemas de ecuaciones lineales vistas en Enseñanzas Básicas. Para ello, se utilizará MATHEMATICA.
2. Conocer los comandos de MATHEMATICA para realizar el tratamiento computacional anteriormente indicado.

Contenidos

1. Factorización LU

Para calcular la factorización LU de una matriz cuadrada A se requieren dos comandos que se usan de manera sucesiva. Uno de estos comandos, debemos programarlos nosotros mismos en Mathematica para usarlo.

El comando `LUdecomposition[A]` devuelve una lista con tres resultados:

1. Primer elemento: forma abreviada de escribir la descomposición LU .
2. Segundo elemento: orden en el que se han considerado las filas, mostrando los intercambios de fila que han sido necesarios.
3. Tercer elemento: número de condición de la matriz de coeficientes A del sistema $A \cdot \underline{x} = \underline{b}$.

Este comando es de suma utilidad debido a la cantidad de información que devuelve. Para poder emplear posteriormente los datos devueltos por el comando, suelen guardarse cada uno como una variable. Veámoslo con un ejemplo:

```
In[1]:={lu,p,nc}=LUdecomposition[ $\begin{pmatrix} 1 & 2 & 1 \\ -1 & 2 & 3 \\ 0 & 1 & 2 \end{pmatrix}$ ]  
Out[1]= {{{1,2,1},{0,1,2},{-1,4,-4}},{1,3,2},1}
```

Aquí la variable `lu` guarda la forma abreviada de la descomposición, la `p` guarda los intercambios de filas y la `cn` corresponde al valor del número de condición del sistema. Obsérvese que, en el caso que nos ocupa, las filas 2 y 3 de la matriz han sido intercambiadas para obtener la factorización LU .

A continuación, hemos de definir el comando `LUMatrices[ ]`:

```
In[2]:=LUMatrices[m_?MatrixQ]:={m - # + IdentityMatrix[Length@m],#}
      &[m*Array[Boole[#<=#2]&,Dimensions@m]]
```

Aplicando este comando sobre la forma abreviada `lu` obtenida anteriormente, obtendremos una lista con las dos matrices L y U de la factorización LU de A :

1. Primer elemento: matriz L .
2. Segundo elemento: matriz U .

Para poder manipular ambas matrices y operar con ellas, el comando suele ejecutarse de tal modo que cada una de las salidas se guarda en una variable diferente. Por ejemplo, podríamos ejecutar la orden siguiente para obtener las matrices L y U :

```
In[3]:={l,u}=LUMatrices[lu]
Out[3]={{{1,0,0},{0,1,0},{-1,4,1}},{{1,2,1},{0,1,2},{0,0,-4}}}
```

En consecuencia, las matrices que se guardan en las variables `l` y `u` son, respectivamente:

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -1 & 4 & 1 \end{pmatrix} \quad \text{y} \quad U = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 2 \\ 0 & 0 & -4 \end{pmatrix}$$

Comentamos anteriormente que para obtener la factorización LU devuelta por Mathematica, el paquete había tenido que intercambiar las filas 2 y 3. Observa que la matriz resultante del producto $L \cdot U$ no es la matriz A , sino la que se obtiene intercambiando dichas filas.

```
In[4]:=l.u
Out[4]={{{1, 2, 1},{0,1,2},{-1,2,3}}
In[5]:=MatrixForm[%]
Out[5]//MatrixForm=
```

$$\begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 2 \\ -1 & 2 & 3 \end{pmatrix}$$

Recuerda que si realizas la factorización LU de la matriz de coeficientes de un sistema y ha sido necesario intercambiar filas, entonces el sistema de ecuaciones sobre el que trabajarás para la factorización LU será el correspondiente a realizar el intercambio de ecuaciones (es decir, incluyendo los términos independientes y no solo la matriz de coeficientes) que haya indicado el intercambio de filas.

2. Factorización de Cholesky

Para calcular la factorización de Cholesky de una matriz cuadrada simétrica y definida positiva, debemos considerar el comando `CholeskyDecomposition`, que se aplicaría sobre la matriz en cuestión, siendo esta su único argumento. La salida que devuelve este comando es la matriz triangular superior $U = L^t$ de la descomposición de Cholesky (que como recordarás era $L \cdot L^t$ con L triangular inferior o, equivalentemente, $U^t \cdot U$ con U triangular superior). Veámoslo con un ejemplo: Definimos una matriz `b` y vemos si es simétrica (i.e. `bt=b`) y definida positiva (i.e. todos los autovalores positivos)

```

In[6]:=b={{3,1,1},{1,4,2},{1,2,5}}
Out[6]={{3,1,1},{1,4,2},{1,2,5}}
In[7]:=Transpose[b]
Out[7]={{3,1,1},{1,4,2},{1,2,5}}
In[8]:=Transpose[b]==b
Out[8]=True
In[9]:=Eigenvalues[b]
Out[9]={Root[-43 + 41 #1 - 12 #1^2 + #1^3 &, 3],
        Root[-43 + 41 #1 - 12 #1^2 + #1^3 &, 2],
        Root[-43 + 41 #1 - 12 #1^2 + #1^3 &, 1]}
In[10]:=N[%]
Out[10]={7.04892, 2.6431, 2.30798}

```

Hemos comprobado que la matriz es igual a su traspuesta (simétrica) y que sus autovalores son positivos (definida positiva). Por tanto, tiene sentido aplicar la descomposición de Cholesky a esta matriz. Veamos cómo se haría esto:

```

In[11]:=u=CholeskyDecomposition[b]
Out[11]={{sqrt(3), 1/sqrt(3), 1/sqrt(3)}, {0, sqrt(11/3), 5/sqrt(33)}, {0, 0, sqrt(43/11)}}
In[12]:=MatrixForm[u]
Out[12]=

$$\begin{pmatrix} \sqrt{3} & \frac{1}{\sqrt{3}} & \frac{1}{\sqrt{3}} \\ 0 & \sqrt{\frac{11}{3}} & \frac{5}{\sqrt{33}} \\ 0 & 0 & \sqrt{\frac{43}{11}} \end{pmatrix}$$


```

Veamos que al realizar el producto correspondiente a la descomposición de Cholesky, aparece nuevamente la matriz original:

```

In[13]:=Transpose[u].u
Out[13]={{3, 1, 1},{1, 4, 2},{1, 2, 5}}
In[14]:=MatrixForm[%]
Out[14]//MatrixForm=

$$\begin{pmatrix} 3 & 1 & 1 \\ 1 & 4 & 2 \\ 1 & 2 & 5 \end{pmatrix}$$


```

Veamos algunos problemas que nos pueden surgir si no aplicamos el comando `CholeskyDecomposition` a una matriz simétrica definida positiva:

1. Si la matriz no es simétrica, nos avisará de ese hecho con el siguiente mensaje:

```

In[15]:=c = {{1, 2, 1}, {1, -1, 2}, {-2, 1, 1}}
Out[15]={{1, 2, 1}, {1, -1, 2}, {-2, 1, 1}}
In[16]:=CholeskyDecomposition[c]
CholeskyDecomposition::herm:
The matrix {{1, 2, 1}, {1, -1, 2}, {-2, 1, 1}} is not Hermitian
or real and symmetric
Out[16]=CholeskyDecomposition[c]

```

2. Si la matriz es simétrica, pero no definida positiva, el aviso será el siguiente:

```
In[17]:=d = {{1, 2, 1}, {2, -5, 1}, {1, 1, 0}}
Out[17]={{1, 2, 1}, {2, -5, 1}, {1, 1, 0}}
In[18]:=CholeskyDecomposition[d]
CholeskyDecomposition::posdef:
The matrix {{1, 2, 1}, {2, -5, 1}, {1, 1, 0}} is not sufficiently
positive definite to complete the Cholesky decomposition to
reasonable accuracy
Out[18]=CholeskyDecomposition[d]
```

3. Normas de vectores y de matrices

En la asignatura de Álgebra ya se estudió el comando `Norm` que permitía calcular el módulo (i.e. la norma euclídea) de un vector. Vamos a revisar este comando para utilizar otras normas distintas a la euclídea y enfatizar que no solo puede aplicarse a vectores, sino también a matrices. Para calcular la norma de un vector v , utilizamos el comando `Norm` con dos argumentos: primero el vector v y segundo el tipo de norma. En concreto, la que viene por defecto seleccionada es la norma 2 (la euclídea), pero podemos calcular otros tipos de norma como la norma 1, la 3 e incluso la ∞ :

```
In[19]:=v={1, 2, 3}
Out[19]={1, 2, 3}
In[20]:=Norm[v] (Norma euclídea)
Out[20]=√14
In[21]:=Norm[v,2] (Norma euclídea)
Out[21]=√14
In[22]:=Norm[v,Infinity] (Norma infinito)
Out[22]=3
```

Pero también puede aplicarse el comando `Norm` a matrices y permite calcular la norma matricial de la matriz indicada como argumento. Veamos algunos ejemplos:

```
In[23]:=Norm[b] (Norma euclídea)
Out[23]=√Root[-1849 + 649#1 - 62#1^2 + #1^3&, 3]
In[24]:=N[%]
Out[24]=7.04892
In[24]:=Norm[d] (Norma euclídea)
Out[24]=√Root[-64 + 185#1 - 38#1^2 + #1^3&, 3]
In[25]:=N[%]
Out[25]=5.6869
In[26]:=Norm[b,Infinity] (Norma infinito)
Out[26]=8
In[27]:=Norm[d,Infinity] (Norma infinito)
Out[27]=8
```

Como podéis comprobar el valor de la norma matricial es distinto de una norma a otra, llegando incluso a ocurrir que dos matrices dan un resultado distinto respecto de una norma y el mismo resultado respecto de otra.

4. Número de condición

Si queremos calcular con Mathematica el número de condición de una matriz, usaremos el comando `LUdecomposition` que calculaba la factorización LU . Recuerda que el tercer dato que devolvía este comando era precisamente dicho valor. Este comando calcula el número de condición para la norma infinito (que es con la que trabajamos en esta asignatura). Veamos un ejemplo:

```
In[28]:=b=MatrixForm[{{7.9, -10.3, 9.5},
{-9.8, 5.8, -4.3},
{10.9, -1.3, -0.6}}]
Out[28]={{7.9, -10.3, 9.5}, {-9.8, 5.8, -4.3}, {10.9, -1.3, -0.6}}
In[29]:=LUdecomposition[b]
Out[29]={{10.9, -1.3, -0.6}, {0.724771, -9.3578, 9.93486},
{-0.899083, -0.494902, 0.0773333}}, {3, 1, 2}, 773.163}
```

Por tanto, el número de condición de la matriz `b` es 773.163. Si queremos calcular el número de condición del sistema sin usar este comando, podemos hacerlo mediante su definición:

$$\text{Número de condición} = \|B\| \cdot \|B^{-1}\|,$$

indicando la norma que a nosotros nos interese en cada caso. De este modo, si escribimos:

```
In[30]:=Norm[b]*Norm[Inverse[b]]
Out[30]= 500.475
```

obtendríamos el número de condición del sistema tomando la norma euclídea. Se observa que el valor es distinto al que salía con el comando `LUdecomposition`. Esto se debe a que este último comando usa la norma infinito (es decir, $\|B\|_\infty \cdot \|B^{-1}\|_\infty$). Por tanto, la orden que debemos escribir para calcular el número de condición del sistema con respecto a la norma infinito sería:

```
In[31]:=Norm[b,Infinity]*Norm[Inverse[b],Infinity]
Out[31]=773.163
```

Calculemos el número de condición para otra matriz más, usando el comando `Norm`:

```
In[32]:=c=MatrixForm[{{1, 2, 1},
{-1, 2, 3},
{0, 1, 2}}]
In[32]:=Norm[c,Infinity]*Norm[Inverse[c],Infinity]
Out[32]=12
```

Obsérvese que el primer ejemplo corresponde a un número de condición elevado (>773), por lo que podríamos afirmar que el sistema está mal condicionado. En el segundo ejemplo, el número de condición es 12 y permite afirmar que el sistema está bien condicionado.

Problemas a realizar

Ejercicio 1. Resuelve el siguiente sistema lineal, que ya viene factorizado usando el método LU :

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} 2 & 3 & -1 \\ 0 & -2 & 1 \\ 0 & 0 & 3 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} 2 \\ -1 \\ 1 \end{pmatrix}.$$

Ejercicio 2. Obtén la factorización LU para las siguientes matrices cuando esto sea posible. Cuando no sea posible reordena las filas de la matriz y obtén la correspondiente factorización:

$$(a) \begin{pmatrix} 1 & 2 & -1 \\ 2 & 4 & 0 \\ 0 & 1 & 1 \end{pmatrix}, \quad (b) \begin{pmatrix} 2.1756 & 4.0231 & -2.1732 & 5.1967 \\ -4.0231 & 6.0000 & 0 & 1.1973 \\ -1.0000 & -5.2107 & 1.1111 & 0 \\ 6.0235 & 7.0000 & 0 & -4.1561 \end{pmatrix}.$$

Ejercicio 3. Resuelve el siguiente sistema de ecuaciones utilizando la factorización de Cholesky:

$$\begin{cases} 3x + y - 3z = 0 \\ -3x + y + 8z = 4 \\ x + 2y + z = 1 \end{cases}$$

Ejercicio 4. Determina una aproximación de la solución del siguiente sistema de ecuaciones mediante los métodos de Jacobi y de Gauss-Seidel, teniendo en cuenta que necesitaríamos 2 cifras decimales exactas y que ha de justificarse la elección de la aproximación inicial $\underline{x}_0$:

$$\begin{cases} 3x + 6y + 2z = 0 \\ 3x - y + z = 1 \\ 3x + 3y + 7z = 4 \end{cases}$$

Ejercicio 5. Obtén una aproximación con tres cifras decimales exactas de la solución del sistema de ecuaciones lineales:

$$\begin{cases} 3x + y - 3z = 0 \\ -3x + y + 8z = 4 \\ x + 2y + z = 1 \end{cases}$$

usando un método de relajación y de tal modo que se asegure tanto la convergencia del método para la elección del iterado inicial $\underline{x}_0$ como una velocidad de convergencia superior a la del método de Gauss-Seidel.

Ejercicio 6. Calcula los números de condición de las siguientes matrices de coeficientes en relación con la norma $\|\cdot\|_\infty$:

$$(a) \begin{pmatrix} \frac{1}{2} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{4} \end{pmatrix}, \quad (b) \begin{pmatrix} 1 & 2 \\ 1.00001 & 2 \end{pmatrix}, \quad (c) \begin{pmatrix} 1.003 & 58.09 \\ 5.550 & 321.8 \end{pmatrix}.$$

Determina si el sistema con estas matrices de coeficientes estaría bien o mal condicionado.

Problemas de ampliación

Ejercicio 7. Obtén la factorización LU para las siguientes matrices cuando esto sea posible. Cuando no sea posible, reordena las filas de la matriz y obtén la correspondiente factorización:

$$(a) \begin{pmatrix} 2 & -1 & 1 \\ 3 & 3 & 9 \\ 3 & 3 & 5 \end{pmatrix}. \quad (b) \begin{pmatrix} 1.012 & -2.132 & 3.104 \\ -2.132 & 4.096 & -7.013 \\ 3.104 & -7.013 & 0.014 \end{pmatrix}.$$

Ejercicio 8. Usando las factorizaciones del Ejercicio 7, resuelve los sistemas lineales cuya matriz de coeficientes es la factorizada en dicho ejercicio y el vector de términos independientes es $\underline{b} = (5, -4, 1)$.

Ejercicio 9. Resuelve, usando la factorización de Cholesky, los siguientes sistemas de ecuaciones lineales:

$$\begin{cases} x + 2y - z = -3 \\ 2x + 6y - z = 2 \\ -x - y + 9z = 0 \end{cases} \quad \begin{cases} -x + 2y - z = -3 \\ 2x - 5y + z = 2 \\ -x + y - 5z = 0 \end{cases}$$

Ejercicio 10. Determina una aproximación con 2 cifras significativas de la solución de los siguientes sistemas de ecuaciones lineales mediante los métodos de Jacobi y de Gauss-Seidel, seleccionando una aproximación inicial $\underline{x}_0$ y asegurando que el método converge para dicha aproximación inicial:

$$(a) \begin{cases} -x + 10y - 2z = 7 \\ 10x - y = 9 \\ -2y + 10z = 6 \end{cases} \quad (b) \begin{cases} x + 4y - z - t = -1 \\ -x - y + 5z + t = 0 \\ 4x + y - z + t = -2 \\ x - y + z + 3t = 1 \end{cases}$$

Ejercicio 11. Aplica un método de sobrerrelajación y otro de subrelajación a los sistemas siguientes para obtener una aproximación con 2 cifras decimales exactas y justificando la convergencia de dicho método para la elección hecha de la aproximación inicial $\underline{x}_0$:

$$\begin{cases} x + 2y - z = -3 \\ 2x + 6y - z = 2 \\ -x - y + 9z = 0 \end{cases} \quad \begin{cases} -x + 2y - z = -3 \\ 2x - 5y + z = 2 \\ -x + y - 5z = 0 \end{cases}$$

Ejercicio 12. Analiza el condicionamiento de los siguientes sistemas lineales:

$$(a) \begin{cases} 3x + y = 7 \\ 3x + 1.0001y = 7.0001 \end{cases} \quad (b) \begin{cases} 2.01x + y + 2z = 5.01 \\ 2x + 3y + 2z = 7 \\ 4x + 16y + 4z = 24 \end{cases}$$

Determina si el sistema está bien o mal condicionado.